CITS3002-Project.pdf
PDF

how should i approach this task

This is a **big systems-style project**, so the worst way to approach it is "start coding randomly." You'll get lost fast. Instead, treat it like building a mini Internet step by step.

Here's a **clear, practical strategy** that aligns directly with the spec.

1. Understand the big picture first

What you are building:

👉 A **simulation (NOT real networking)**
👉 Data flows: **Application → L4 → L3 → L2 → Router → L2 → L3 → L4 → Application**
👉 Then ACK comes back the same way CITS3002-Project

So think of it as:

- No sockets
- No real packets
- Just **Python objects passed between functions**

2. Design before coding (this is critical)

The project already tells you the structure:

- `main.py` → runs everything
- `protocol.py` → headers + layers
- `devices.py` → Host, Router
- `config.py` → IP, MAC, routing tables CITS3002-Project

Your mental model should look like:

```
Application
   ↓
Transport Layer (L4)
```

```
   ↓
Network Layer (L3)
   ↓
Data Link Layer (L2)
   ↓
Device (Host / Router)
```

3. Build from bottom-up (NOT top-down)

Most students try L4 first and suffer. Don't.

Step 1 — Data Structures (easy win)

Create classes for:

- `Frame` (L2)
- `Packet` (L3)
- `Segment` (L4)

Just store fields, no logic yet.

Step 2 — Layer 2 (start here)

Why? It's simplest and everything depends on it.

Implement:

- Frame creation
- MAC table (dictionary)
- Send/receive functions

Key idea:




```
mac_table[ip] = mac
```

Focus on:

- Encapsulation
- Forwarding
- Logging

Step 3 — Layer 3 (routing logic)

Now add:

- Routing table lookup
- TTL decrement
- Next-hop decision CITS3002-Project

Core logic:

Python Run

```python
if dst_ip in same subnet:
    next_hop = dst_ip
else:
    next_hop = router_ip
```

Step 4 — Layer 4 (hardest part)

This is where marks are:

You must implement **rdt2.2 (alternating bit protocol)** CITS3002-Project

Key ideas:

- Sequence number: `0 ↔ 1`
- Wait for ACK before sending next
- Retransmit on wrong ACK

Basic flow:

Python Run

```python
send DATA(seq=0)
wait ACK(0)
send DATA(seq=1)
```

Step 5 — Devices (Host & Router)

Host:

- Starts communication
- Handles L4 logic (send/receive)

Router:

- ONLY L2 + L3

- No L4 logic

4. Build the pipeline (most important part)

You must connect layers like this:

Example (Host A sending):

Python Run

```
L4.send(data)
→ L3.encapsulate()
→ L2.encapsulate()
→ send to Router
```

Router:

Python Run

```
L2.receive()
→ L3.route()
→ L2.send()
```

5. Logging = free marks (don't ignore this)

The spec literally gives you the expected output format CITS3002-Project

👉 Mimic it closely
👉 Same wording style

Markers WILL check this.

6. Handle segmentation early

Constraint:

- Max segment size = 500 bytes CITS3002-Project

So:

Python Run

```
if data > 500:
```

```
split into chunks
```

Then send **one-by-one with ACK**

7. Suggested implementation order (very important)

Follow this exactly:

1. ✅ Data classes (Frame, Packet, Segment)
2. ✅ Layer 2 send/receive
3. ✅ Layer 3 routing
4. ✅ Basic end-to-end WITHOUT ACK
5. ✅ Add ACK logic (rdt2.2)
6. ✅ Add segmentation
7. ✅ Polish logging

8. Common mistakes (avoid these)

❌ Mixing layers (keep responsibilities separate)
❌ Router doing transport logic
❌ Forgetting TTL decrement
❌ Skipping MAC table learning
❌ Implementing real networking (NOT required)
❌ Doing everything in one file

9. What a good solution looks like

- Clean separation of layers
- Functions like:
  - `send_segment()`
  - `route_packet()`
  - `forward_frame()`
- Logs clearly showing:
  - encapsulation
  - forwarding
  - ACK handling

10. If you want a high mark

Focus on:

- Clean architecture (easy 10 marks)
- Correct rdt2.2 (hard but high weight)
- Matching logs exactly

If you want next step

I can:

- Design a **full class diagram**
- Give you a **starter code template**
- Or walk you through **rdt2.2 implementation (this is the tricky part)**

Just tell me 👍

Sources